

Arquitetura Event-Driven — Pinduoduo Clone

Laboratório 5 | PUC Minas | Pedro Henriue Barbosa Montandon de Araújo | Maio 2026

VISÃO GERAL

O sistema implementa um clone do Pinduoduo (plataforma de compras coletivas) seguindo arquitetura orientada a eventos com Domain-Driven Design (DDD). A aplicação é um monólito modular em NestJS, onde três módulos de domínio se comunicam de forma assíncrona por meio do RabbitMQ, garantindo baixo acoplamento e responsabilidades bem delimitadas por contexto de negócio.

ESCOLHAS DE TECNOLOGIA

Camada	Tecnologia
Framework	NestJS 10 (TypeScript)
Broker de eventos	RabbitMQ 3 — topic exchange
Banco de dados	PostgreSQL 16 + TypeORM
Autenticação	JWT + Passport + bcrypt
Agendamento	@nestjs/schedule (Cron)
Infraestrutura	Docker Compose
Testes	Jest — unit e integration

Por que RabbitMQ? Topic exchanges roteiam cada evento por chave (ex.: `group_purchase.created`), permitindo múltiplos consumidores sem alterar o produtor. Filas duráveis garantem entrega mesmo com consumidor offline.

FLUXO DE EVENTOS

Criação: `POST /group-purchases` → `GroupPurchase.create()` → `[group_purchase.created]` → RabbitMQ exchange `domain_events` → `ProductConsumer` reserva estoque

Confirmação (mínimo atingido): `POST /group-purchases/:id/join` → `GroupPurchase.addParticipant()` → `[group_purchase.confirmed]` → `UserConsumer` notifica participantes

Expiração (automática): CRON a cada 1 min → `GroupPurchase.expire()` → `[group_purchase.expired]` → `ProductConsumer` libera estoque reservado

DECISÕES ARQUITETURAIS

- Monólito modular: único deploy com limites lógicos claros entre módulos — facilita migração futura para microsserviços.
- Agregados DDD: `GroupPurchase` encapsula regras de negócio e gera eventos internamente; use case apenas persiste e publica.
- Topic exchange: roteamento por routing key — produtores não conhecem consumidores, adicionando novas assinaturas sem alterar código existente.
- Filas duráveis: mensagens persistidas no broker, sem perda em reinicializações ou falhas transientes do consumidor.
- Repository pattern: acesso a dados abstraído por tokens de injeção (Symbol) — troca de banco sem afetar domínio.
- Stateless auth: JWT sem sessão em servidor, escalável horizontalmente.
- Testes de integração: validam fluxo completo de eventos com aplicação real e RabbitMQ, evitando falsos positivos de mocks.

MÓDULOS DE DOMÍNIO

Group Purchase (produtor central)

- Cria e gerencia campanhas de compra coletiva
- Confirma campanha ao atingir participantes mínimos
- Job CRON expira campanhas vencidas a cada minuto
- Publica 5 tipos de eventos de domínio

Product (consumidor — estoque)

- Reserva estoque em `group_purchase.created`
- Libera estoque em `group_purchase.expired`

User (consumidor — notificações)

- Notifica participantes em `group_purchase.confirmed`

Branch: `event-driven` | Infraestrutura: `docker-compose.yml` (PostgreSQL + RabbitMQ) | Eventos: `created` · `participant_joined` · `participant_left` · `confirmed` · `expired`